



SYSTÈMES EMBARQUÉS NUMÉRIQUES

SEI – SoC

---

# Implémentation HW/SW d'un CNN sur cible lowRISC/RocketCHIP

---

***Auteurs :***

Martin LAURENT  
Jérémy CHAGNY  
Milane DAOUDI  
Ordan ONOKOMBA

***Enseignant :***

Mounir BENABDNEBI

# Table des matières

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>2</b>  |
| 1.1      | Contexte . . . . .                                                     | 2         |
| 1.2      | Objectifs . . . . .                                                    | 2         |
| 1.3      | Organisation du rapport . . . . .                                      | 2         |
| 1.4      | Organisation du travail . . . . .                                      | 2         |
| <b>2</b> | <b>Prise en main du système</b>                                        | <b>3</b>  |
| 2.1      | Architecture globale . . . . .                                         | 3         |
| 2.1.1    | Flot HW/SW . . . . .                                                   | 4         |
| <b>3</b> | <b>Méthodologie de test et validation</b>                              | <b>5</b>  |
| 3.1      | Stratégie de test . . . . .                                            | 5         |
| 3.2      | Critères de validation . . . . .                                       | 5         |
| <b>4</b> | <b>Développement des fonctions</b>                                     | <b>7</b>  |
| 4.1      | Mise en mémoire des images depuis la carte SD . . . . .                | 7         |
| 4.2      | Contrôle de l'écran . . . . .                                          | 7         |
| 4.3      | Filtre Sobel . . . . .                                                 | 8         |
| 4.4      | Implémentation du réseau de neurones convolutif (CNN) . . . . .        | 8         |
| 4.4.1    | Prétraitement des images . . . . .                                     | 8         |
| 4.4.2    | Architecture du réseau . . . . .                                       | 8         |
| 4.4.3    | Implémentation logicielle . . . . .                                    | 9         |
| 4.4.4    | Test sur PC de développement . . . . .                                 | 9         |
| 4.4.5    | Intégration dans l'application embarquée . . . . .                     | 10        |
| 4.5      | Gestion des entrées utilisateurs . . . . .                             | 10        |
| <b>5</b> | <b>Résultats</b>                                                       | <b>11</b> |
| 5.1      | Fonctionnalité . . . . .                                               | 11        |
| 5.2      | Classification sur quelques images CIFAR-10 . . . . .                  | 11        |
| 5.3      | Mesure du temps par lecture du registre <code>rdcycle</code> . . . . . | 12        |
| <b>6</b> | <b>Difficultés rencontrées</b>                                         | <b>14</b> |
| <b>7</b> | <b>Améliorations possibles</b>                                         | <b>15</b> |
| 7.1      | Améliorations logicielles . . . . .                                    | 15        |
| 7.2      | Améliorations algorithmiques . . . . .                                 | 15        |
| 7.3      | Améliorations matérielles . . . . .                                    | 15        |

# 1 Introduction

## 1.1 Contexte

Les systèmes embarqués modernes intègrent de plus en plus de fonctionnalités de traitement d'images et de vision artificielle, telles que la détection de contours ou la classification par réseaux de neurones convolutifs (CNN). Ces algorithmes présentent toutefois un coût computationnel élevé et une forte dépendance à la bande passante mémoire, ce qui rend leur déploiement sur des architectures embarquées particulièrement contraignant.

Dans ce contexte, les plateformes open-source basées sur RISC-V, telles que Rocket Chip et les développements proposés par la fondation lowRISC, constituent un environnement pertinent pour étudier concrètement les problématiques de co-conception matériel/logiciel. L'utilisation d'un processeur softcore implémenté sur FPGA permet d'explorer les interactions entre architecture matérielle, logiciel embarqué et performances applicatives.

Ce projet s'inscrit dans cette démarche et consiste à réaliser un démonstrateur complet intégrant la lecture d'images depuis une carte SD, leur prétraitement, leur affichage sur écran VGA, l'application de filtres (Sobel et CNN), ainsi que l'interaction utilisateur via interruptions matérielles.

## 1.2 Objectifs

Les objectifs principaux du projet sont :

- prendre en main une plateforme RISC-V softcore générée avec Rocket Chip et intégrée sur FPGA ;
- développer une application bare-metal intégrant la lecture d'images depuis une carte SD et leur affichage VGA ;
- implémenter des traitements d'images embarqués, notamment un filtre de Sobel et un réseau de neurones convolutif CIFAR-10 ;
- mettre en œuvre la gestion des interruptions via le contrôleur PLIC pour assurer l'interaction utilisateur par boutons poussoirs ;
- analyser les performances obtenues et identifier les limitations liées à l'architecture et aux choix d'implémentation.

## 1.3 Organisation du rapport

Ce rapport présente successivement l'architecture matérielle et logicielle du système développé, les différentes étapes d'implémentation logicielle (lecture SD, prétraitement, filtres d'images, CNN, affichage VGA, gestion des interruptions), puis les résultats expérimentaux, les performances mesurées, les difficultés rencontrées et les perspectives d'amélioration.

## 1.4 Organisation du travail

Le projet a été réalisé sur une durée de deux semaines selon une organisation progressive. La première semaine a été consacrée à la prise en main de la plateforme lowRISC, à l'analyse du code existant et à l'implémentation des fonctionnalités de base, notamment la lecture des images depuis la carte SD et leur pré-traitement. La seconde semaine a permis d'intégrer les traitements plus avancés tels que l'affichage VGA, le filtrage par convolution,

l'implémentation du CNN ainsi que la gestion des interruptions via les boutons poussoirs. Cette organisation incrémentale a facilité le débogage et la validation fonctionnelle de chaque étape.

Le diagramme de Gantt suivant décrit l'organisation générale du projet.

## Organisation projet Systèmes embarqués numériques

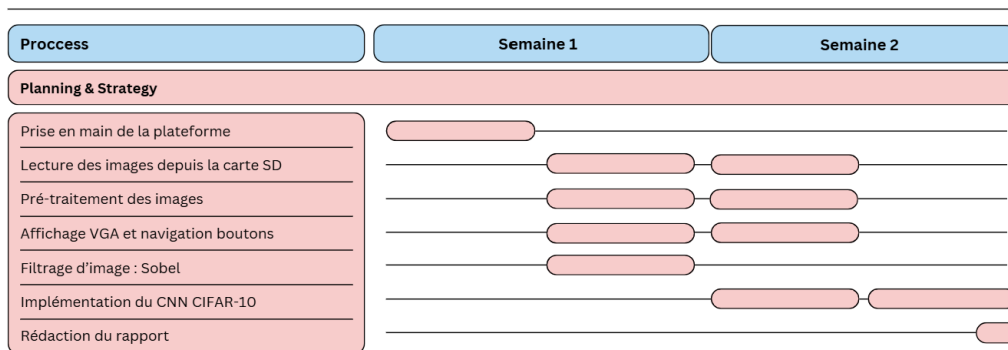


FIGURE 1 – Diagramme de Gantt représentant l'organisation du projet sur deux semaines

## 2 Prise en main du système

### 2.1 Architecture globale

Le système repose sur une carte Nexys A7, sur laquelle est implémenté un softcore RISC-V RV64IMAFD généré à partir de l'architecture Rocket Chip. Le processeur s'appuie sur une mémoire BRAM interne de 4 860 KB, utilisée pour stocker le code, les images et les données intermédiaires nécessaires aux traitements.

Plusieurs IP matérielles sont intégrées au système afin d'assurer l'interfaçage avec les périphériques externes. Un contrôleur SD card permet la lecture des images depuis la carte mémoire, tandis qu'un contrôleur VGA assure l'affichage des images et des résultats de traitement sur un écran. La communication série est réalisée via un UART, principalement utilisé pour le débogage. La gestion des interruptions est assurée par le PLIC, auquel sont connectés les boutons poussoirs, permettant une interaction utilisateur pour la sélection des images et des filtres.

La partie logicielle repose sur une architecture en deux niveaux. Un bootloader, situé à l'adresse 0x4000\_0000, initialise le système et lance l'exécution de l'application principale. L'application, chargée à l'adresse 0x8000\_0000, implémente l'ensemble des fonctionnalités du démonstrateur : lecture des images depuis la carte SD, prétraitement des images, application de filtres (détection de contours de type Sobel et classification par CNN), affichage VGA des résultats et gestion des interruptions utilisateur.

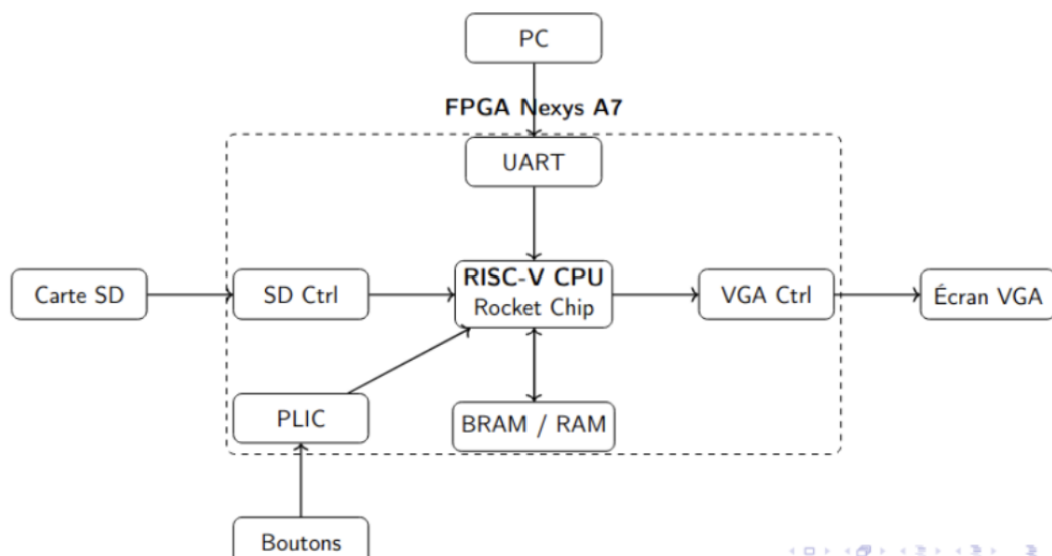


FIGURE 2 – Architecture globale du système

### 2.1.1 Flot HW/SW

Lorsque nous avons effectué la démonstration 0 (issue de Technical\_documentation.pdf), nous avons chargé le bitstream dans la FPGA. Nous avons vérifié que le cross-compilateur permettait de compiler des commandes assembleurs sur opérateurs flottants et que la configuration hardware contenait bien une unité de calculs pour opérateurs flottants. Après cela, nous avons interagi uniquement avec le flot SW qui est nettement plus simple à utiliser. C'est lors de cette étape que nous avons modifié le Makefile en s'inspirant des erreurs affichées pour résoudre les conflits. Une fois cette étape mise en place, nous avons pu avancer vers les tâches de développement.

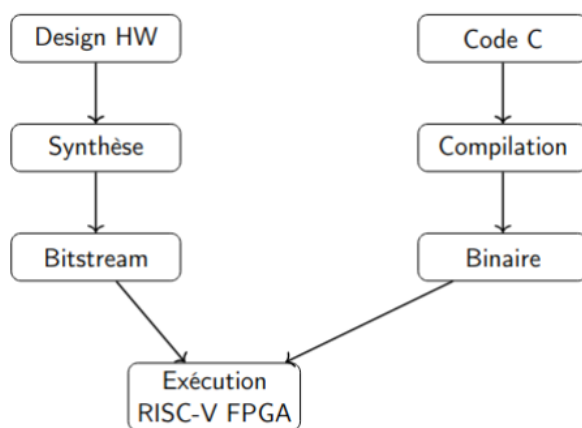


FIGURE 3 – Flot HW/SW

## 3 Méthodologie de test et validation

### 3.1 Stratégie de test

Pour être efficace dans le développement de notre CNN sur FPGA, une stratégie de test a été définie. Chaque module fonctionnel a d'abord été validé indépendamment avant d'être intégré progressivement dans le système.

La première étape a été de mettre en place des tests unitaires pour chacune des étapes majeures du projet. Pour cela, plusieurs fichiers `application.c` ont été développés de manière à avoir un environnement de tests dédiés. Ces tests ont permis de valider les fonctionnalités indépendamment des autres, comme l'affichage VGA, le preprocessing des images, les filtres et les étapes du CNN. Les résultats intermédiaires ont pu être vérifiés à l'aide de `print` dans le terminal, ainsi que par des observations visuelles à l'écran en les comparant avec des images de référence.

Une fois chaque module validé, il a été question de mettre en place une phase d'intégration progressive. Plusieurs "sous-systèmes" fonctionnels ont été assemblés en veillant à la cohérence des données transférées entre les deux. Cette méthode a permis de diagnostiquer efficacement les erreurs d'interface ou de synchronisation entre les modules.

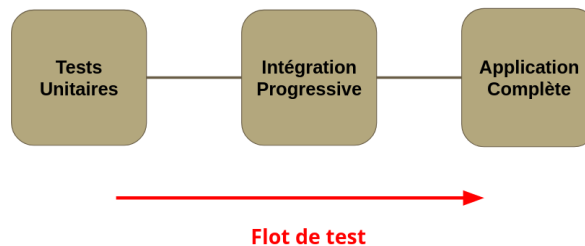


FIGURE 4 – Stratégie de test

Enfin, le système au complet a été testé en conditions réelles. Notamment en activant les boutons. Cette étape a permis de valider la conception de notre système.

### 3.2 Critères de validation

La validation du système repose sur plusieurs critères complémentaires qui permettent d'évaluer à la fois la fonctionnalité et les performances de l'application.

Le premier critère est l'exactitude fonctionnelle. Il faut vérifier que chaque fonctionnalité du système produit les résultats attendus. Cela implique la bonne prise en compte des entrées utilisateurs, la sélection correcte des images et des filtres, mais aussi la validité des résultats des traitements d'image et de la reconnaissance d'image par le CNN.

Le second critère est basé sur la latence. Notamment, la réactivité du système face à une action utilisateur consiste à mesurer le temps écoulé entre la pression d'un bouton et la mise à jour de l'affichage.

Enfin, l'utilisation des ressources constitue le dernier critère de validation. Le mappage de la mémoire, la charge du processeur et l'impact des différents traitements auraient dû être analysés. Malheureusement, les différents prints ne s'affichent pas correctement dans le terminal, il a été difficile de récupérer certaines valeurs (nombre de cycles, mappage mémoire...)

## 4 Développement des fonctions

### 4.1 Mise en mémoire des images depuis la carte SD

La mise en mémoire des images constitue la première étape du traitement. Les images sont stockées sur une carte SD au format PPM (P3) et sont lues séquentiellement par le processeur RISC-V. Pour chaque image, le programme ouvre le fichier correspondant, vérifie son format et extrait les informations de résolution afin de s'assurer de leur compatibilité avec le système d'affichage et de traitement.

Les valeurs des pixels RGB sont ensuite lues et stockées dans un tableau global. Les images sont organisées de manière contiguë en mémoire, ce qui permet un accès efficace lors des étapes suivantes de pré-traitement, d'affichage ou de classification. Cette organisation évite de relire les données depuis la carte SD et améliore ainsi les performances globales du système.

### 4.2 Contrôle de l'écran

L'affichage des résultats du CNN est piloté par un **contrôleur VGA matériel** (déjà présent) sur le FPGA. Le processeur écrit dans un **framebuffer mémoire**, lu en continu par le contrôleur pour générer les signaux VGA.

- **Pixel** : 8 bits (niveaux de gris)
- **Accès mémoire** : par mots de 64 bits (8 pixels/mot)
- **Résolution** :  $640 \times 480$  pixels
- **Adressage** :  $\text{index} = x + y \times \frac{640}{8}$  (x par groupe de 8 pixels, y ligne)

**Fonction display** : Interface logicielle appelée après le traitement par le CNN. Elle :

1. Sélectionne le **mode d'affichage** (BYPASS, CNN\_CLASSIFIER, EDGE\_DETECTOR)
2. Transmet la classe prédite
3. Appelle `on_screen` pour l'écriture

**Fonction on\_screen** : Écrit effectivement dans le framebuffer en combinant :

1. L'image traitée (mémoire)
2. Un **overlay graphique** (pictogramme de classe, stocké en ROM)

L'écriture se fait ligne par ligne, par mots de 64 bits.

#### Synchronisation par signaux Done

Pour éviter les recalculs et garantir la cohérence :

- Des états persistants (`edgeDone[]`, `cnnDone[]`) indiquent si un traitement (filtrage, CNN) est **terminé** pour une image donnée
- L'affichage n'est déclenché que lorsque l'état correspondant est actif
- Le contrôleur VGA lit le framebuffer de manière autonome ; aucune gestion logicielle des signaux de synchronisation n'est nécessaire

**Remarque** : L'activation de l'affichage VGA rend le canal série (e.g., `printf` via `picocom`) indisponible pour le débogage.



### 4.3 Filtre Sobel

Le filtre de Sobel a été implémenté afin de réaliser une détection de contours sur les images en niveaux de gris affichées en  $640 \times 480$ . Il repose sur une convolution 2D utilisant un noyau de taille  $3 \times 3$  permettant d'estimer le gradient d'intensité des pixels.

L'algorithme parcourt chaque pixel de l'image d'entrée, applique le noyau de convolution, puis calcule la valeur absolue du résultat afin d'obtenir l'amplitude du contour. Une normalisation simple est ensuite effectuée pour améliorer le contraste de l'image filtrée avant son affichage.

Les images filtrées sont stockées dans un tableau global `TAB_GS_FILTERED`, afin d'éviter de recalculer le filtre à chaque affichage.

Cette implémentation, bien que fonctionnelle, reste perfectible, notamment par l'utilisation de calculs en virgule fixe, l'optimisation des boucles ou encore une meilleure gestion des bords de l'image.

### 4.4 Implémentation du réseau de neurones convolutif (CNN)

Un réseau de neurones convolutif inspiré de l'architecture CIFAR-10 a été implémenté afin de réaliser la classification d'images directement sur le processeur RISC-V embarqué. L'objectif est d'associer à chaque image affichée une classe parmi les 10 catégories du jeu de données CIFAR-10.

#### 4.4.1 Prétraitement des images

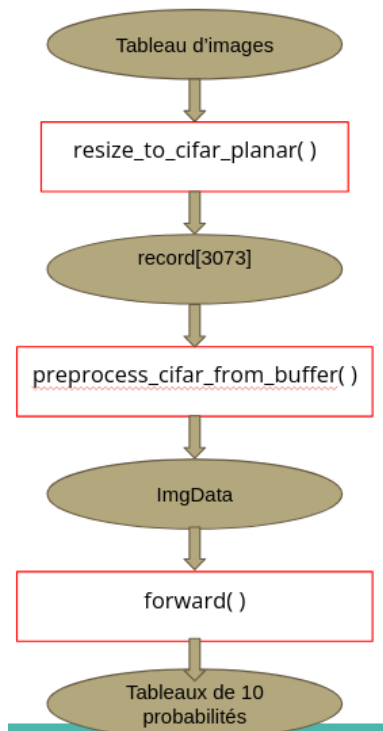


FIGURE 5 – Architecture du `main()`

Les images sources en format RGB  $640 \times 480$  sont d'abord redimensionnées en  $32 \times 32$  par la méthode du *nearest-neighbor*, puis converties en format planaire (canaux R, G, B séparés). Un recadrage central (*center-crop*) est ensuite appliqué afin d'obtenir des images de taille  $24 \times 24 \times 3$ .

Les pixels sont convertis en flottants dans l'intervalle  $[0, 1]$ , puis normalisés (moyenne nulle, variance unitaire) avant d'être injectés dans le réseau. Ces opérations sont réalisées par les fonctions `resize_to_cifar_planar()` et `preprocess_cifar_from_buffer()`.

#### 4.4.2 Architecture du réseau

Le CNN implémenté est constitué des étapes suivantes :

- Trois blocs successifs **Convolution + ReLU + MaxPooling**
- Une opération de **flatten** transformant les tenseurs 3D en vecteur 1D
- Une couche **fully-connected** (perceptron)
- Une couche **softmax** produisant les probabilités par classe

Les convolutions utilisent un *padding "same"* afin de conserver les dimensions spatiales, tandis que les opérations de MaxPooling réduisent progressivement la résolution des cartes de caractéristiques.



FIGURE 6 – Architecture du CNN

#### 4.4.3 Implémentation logicielle

Le réseau est implémenté intégralement en langage C dans le dossier `cnn_riscv_mancini`. Chaque type d'opération (convolution, ReLU, maxpool, flatten, dense, softmax) est réalisé par une fonction dédiée.

La fonction principale `perform_cnn(int number)` orchestre l'enchaînement des différentes couches et retourne :

- le vecteur de probabilités de taille 10,
- la classe prédite correspondant à la probabilité maximale.

Une gestion explicite de la mémoire est utilisée : chaque couche alloue dynamiquement ses sorties et libère les entrées devenues inutiles afin de limiter l'empreinte mémoire sur le système embarqué.

#### 4.4.4 Test sur PC de développement

Avant intégration dans l'application embarquée, nous avons reproduit l'environnement de l'application embarquée sur le PC de développement. C'est à dire que le code a accès

uniquement à un tableau initialisé de coefficients et aux pixels de l'image directement dans un tableau aussi. Ainsi, on a reproduit exactement le même environnement de manière à faciliter le passage du PC de développement à l'application embarquée.

```
Image 997 : true=1, pred=8
Image 998 : true=3, pred=2
Image 999 : true=8, pred=8

=====
Accuracy on 1000 images = 62.70%
=====
```

FIGURE 7 – Résultats du CNN sur PC de développement

#### 4.4.5 Intégration dans l'application embarquée

La fonction `perform_cnn()` a été ajoutée dans `application.c`. Elle réalise l'ensemble du pipeline :

1. sélection de l'image courante,
2. redimensionnement et normalisation,
3. exécution du CNN,
4. récupération de la classe prédite.

Lorsque le filtre `CNN_CLASSIFIER` est sélectionné, cette fonction est appelée par la routine d'affichage. Le label correspondant à la classe prédite est alors affiché en surimpression sur l'image VGA.

Afin de préserver la réactivité du système, les résultats du CNN sont mis en cache et calculés une seule fois par image. L'image brute est affichée immédiatement, tandis que le résultat de classification est superposé dès qu'il est disponible.

### 4.5 Gestion des entrées utilisateurs

La gestion des boutons du FPGA repose sur une chaîne d'interruption. Les boutons physiques sont reliés à un périphérique qui génère les interruptions externes lorsque l'un d'entre eux est pressé. Ces interruptions sont transmises au PLIC qui est chargé de prioriser et distribuer les interruptions aux CPU.

Lorsque le PLIC reçoit l'information qu'un bouton est actionné, il déclenche une interruption externe vers le processeur. Cette interruption est traitée en mode machine. Le processeur interrompt alors l'exécution du programme principal pour exécuter la routine du service d'interruption.

Cette architecture permet de découpler efficacement la détection des événements sans trop surcharger le CPU.

Les routines d'interruptions constituent le point d'entrée du software lors de l'interruption matérielle. Le rôle principal de l'ISR est d'identifier quel bouton a été pressé puis de mettre à jour deux variables globales :

- **imageSel**, qui détermine l'image qui est affichée à l'écran
- **filterSel**, qui sélectionne le mode de traitement appliqué (Bypass, Edge Detector, CNN...)

Aucun traitement lourd n'est fait directement au sein de l'ISR, les traitements ou les calculs sont réalisés dans le `main()`. Ensuite, l'ISR acquitte l'interruption auprès du PLIC et rend la main au programme principal.

L'avantage de cette approche réside dans faible latence de réaction aux interruptions et le faible temps passé en contexte d'interruption. Cela garantit une certaine performance de notre système et en particulier du CNN.

## 5 Résultats

### 5.1 Fonctionnalité

L'ensemble des fonctionnalités définies dans le cahier des charges a été implémenté et validé sur la plateforme FPGA. Le système final permet :

- la lecture correcte des images depuis la carte SD ;
- leur stockage en mémoire et leur prétraitement ;
- l'affichage en temps réel sur écran VGA ;
- l'application du filtre de Sobel pour la détection de contours ;
- l'exécution du réseau de neurones convolutif pour la classification ;
- l'interaction utilisateur via les boutons poussoirs (sélection d'image et de filtre) à l'aide d'interruptions matérielles.

Chaque fonctionnalité a d'abord été testée indépendamment à l'aide de programmes dédiés, puis validée lors de la phase d'intégration globale. Les transitions entre les différents modes d'affichage (bypass, détection de contours, classification CNN) sont fonctionnelles et stables, et les résultats affichés correspondent aux traitements appliqués.

Ainsi, le démonstrateur final respecte intégralement les spécifications fonctionnelles initiales et répond aux objectifs fixés pour ce projet.

### 5.2 Classification sur quelques images CIFAR-10

Des tests de classification ont été réalisés sur un sous-ensemble d'images issues du jeu de données CIFAR-10 afin d'évaluer le bon fonctionnement du CNN embarqué.

Les résultats obtenus sur la plateforme FPGA sont cohérents avec ceux mesurés sur le PC de développement : on retrouve les mêmes classes prédites ainsi que des pourcentages de confiance (probabilités issues de la couche softmax) très proches, à de légères différences numériques près dues aux conditions d'exécution et aux optimisations du compilateur.

Autrement dit, le passage de l'environnement PC à l'environnement embarqué n'a pas dégradé le comportement du réseau. Le modèle conserve la même capacité de discrimination et produit des résultats équivalents en termes de précision de classification.

Ces tests confirment :

- la validité de l'implémentation logicielle du CNN sur l'architecture RISC-V ;
- la cohérence du prétraitement des images entre les deux plateformes ;
- la reproductibilité des résultats malgré les contraintes matérielles du système embarqué.

### 5.3 Mesure du temps par lecture du registre `rdcycle`

La mesure du temps d'exécution est réalisée à l'aide du registre matériel `cycle`, accessible via l'instruction RISC-V `rdcycle`. Ce registre est un compteur 64 bits incrémenté à chaque cycle d'horloge du processeur.

Le principe consiste à lire la valeur du compteur avant et après l'exécution d'une fonction, puis à calculer la différence entre ces deux valeurs afin d'obtenir le nombre de cycles consommés.

**Code utilisé :** Le code suivant montre l'implémentation logicielle permettant de lire le registre `cycle` depuis le programme C :

```
static inline uint64_t rdcycle(void)
{
    uint64_t cycles;
    asm volatile ("rdcycle %0" : "=r"(cycles));
    return cycles;
}
```

Cette fonction encapsule l'instruction assembleur `rdcycle`, qui copie le contenu du registre matériel dans un registre général du processeur.

**Méthode de mesure :** La mesure du nombre de cycles s'effectue comme suit :

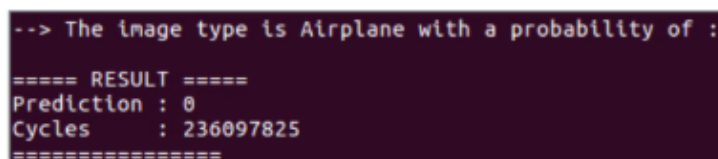
```
uint64_t start = rdcycle();
perform_cnn(img_number); // Code à mesurer
uint64_t end = rdcycle();

uint64_t cycles = end - start;
```

Le temps d'exécution est ensuite déduit à partir de la fréquence du processeur :

$$T = \frac{N_{cycles}}{f_{on\_board}}, \quad \text{avec } f_{on\_board} = 100 \text{ MHz.}$$

**Résultats expérimentaux** La Figure 8 présente le nombre de cycles mesuré pour l'exécution complète du CNN sur une image. Les mesures ont été réalisées après une phase de *warm-up* afin d'initialiser les caches et le pipeline du processeur.



```
--> The image type is Airplane with a probability of :
===== RESULT =====
Prediction : 0
Cycles      : 236097825
=====
```

FIGURE 8 – Nombre de cycles mesurés pour l'exécution du CNN

### Remarques

- Le compteur `rdcycle` étant directement lié à l'horloge `on_board`, la mesure est indépendante du système d'exploitation et ne nécessite aucune interruption.
- Cette méthode offre une précision au cycle près.
- Les interruptions ainsi que l'affichage VGA ont été désactivés durant la mesure afin d'éviter toute perturbation du comptage.

## 6 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées tout au long du projet, tant sur les aspects logiciels que matériels.

**Variabilité matérielle et communication** Des variations de comportement ont été observées selon le port USB utilisé pour la programmation et la communication série, ce qui a parfois compliqué la reproductibilité des tests. Il faut absolument utiliser le port USB en bas à gauche si on veut espérer voir des traces lisibles dans un terminal picocom.

**Fichiers à sourcer et ordre** Si l'on veut déboguer via gdb et voir des prints lisibles dans un terminal picocom, il faut (depuis la machine virtuelle) :

- Ouvrir un terminal et sourcer `set_env.sh` puis `sourceme.sh` puis faire `make debug` dans le terminal en ayant préalablement connecté le FPGA au bon port USB et activé le device dans l'interface de la VM depuis périphériques -> USB
- Ouvrir un second terminal dans lequel on exécute l'exécutable riscv avec `./riscv64-unknown-elf...` et NE RIEN SOURCER
- Ouvrir un 3ème terminal pour picocom.

**Complexité de l'environnement lowRISC** L'arborescence du projet lowRISC ainsi que le nombre important de fichiers de configuration et de scripts ont rendu la prise en main initiale délicate. L'ordre de compilation et de liaison des différents modules devait être strictement respecté, sous peine d'erreurs difficiles à diagnostiquer.

**Débogage en environnement bare-metal** Le développement sans système d'exploitation a fortement compliqué le débogage. L'utilisation de `printf()` était limitée, notamment lorsque le contrôleur VGA était activé, ce qui empêchait parfois l'utilisation conjointe du débogage via GDB. Il a donc été nécessaire de recourir à des traces UART minimales et à des tests unitaires progressifs.

Ces difficultés ont néanmoins permis d'approfondir la compréhension des systèmes embarqués réels et des problématiques liées à la co-conception matériel/logiciel.

## 7 Améliorations possibles

Bien que le démonstrateur réponde au cahier des charges fonctionnel, plusieurs axes d'amélioration peuvent être envisagés afin d'optimiser les performances, la consommation de ressources et la qualité globale du système.

### 7.1 Améliorations logicielles

- **Passage en virgule fixe** : remplacer les calculs en virgule flottante par des opérations en virgule fixe permettrait de réduire significativement le temps d'exécution du CNN et du filtre Sobel.
- **Suppression des allocations dynamiques** : l'utilisation de buffers statiques préalloués éviterait les appels répétés à `malloc/free`, coûteux en temps et sources potentielles de fragmentation mémoire.
- **Réduction des accès mémoire** : une meilleure organisation des données (localité mémoire, fusion de boucles, mise en cache logicielle) permettrait de diminuer la latence des traitements.

### 7.2 Améliorations algorithmiques

- **Quantification des poids du CNN** : l'utilisation de poids sur 8 ou 16 bits permettrait de réduire considérablement les besoins mémoire et le temps de calcul.
- **Architecture CNN plus compacte** : un réseau moins profond ou avec moins de filtres pourrait offrir un compromis intéressant entre précision et performances.
- **Redimensionnement plus efficace** : l'adoption d'algorithmes plus rapides ou approximatifs pourrait réduire le temps de prétraitement.

### 7.3 Améliorations matérielles

- **Accélérateur matériel pour les convolutions** : l'intégration d'un coprocesseur dédié ou d'un accélérateur FPGA permettrait de réduire drastiquement le temps d'inférence du CNN.
- **Pipelining du flux de pixels** : le traitement en flux continu entre la lecture SD, le prétraitement et l'affichage VGA améliorerait la latence globale.
- **Utilisation d'un DMA** : un contrôleur DMA permettrait de décharger le processeur des transferts mémoire volumineux, notamment pour l'accès aux images et aux buffers intermédiaires ainsi que pour les coefficients.

Ces perspectives illustrent le potentiel d'évolution du projet vers un système embarqué plus performant et plus proche des architectures utilisées dans les applications industrielles de vision embarquée.